FASE DE DESARROLLO DEL SISTEMA DE INVENTARIOS MRO

Caso de estudio: YPFB Transporte S.A.

Construcción del prototipo funcional y evidencia de su funcionamiento

Tatiana Aramayo, Ariana Cordero, Patricia Quisbert y Melany Sonco
Sistemas de Información Gerencial · Facultad de Ingenierías y Arquitectura · Universidad Privada Boliviana
Octubre de 2026

La fase de desarrollo convierte los modelos de las fases anteriores en un sistema que funciona. Se construyó un prototipo con las tres capas de la propuesta tecnológica: una aplicación web en React, una API de negocio en Node.js con Express, un trabajador de integración, una base de datos PostgreSQL de 23 tablas y una cola de operaciones en RabbitMQ. El WMS y el ERP de YPFB Transporte S.A. son sistemas existentes; en el prototipo los reemplazan dos simuladores que exponen el mismo contrato de integración, de modo que el sistema se prueba con reservas, salidas, órdenes de compra, recepciones y fallas reales de comunicación.

El prototipo es básico en alcance, pero aplica todos los conceptos de inventario definidos en la fase conceptual: registro perpetuo, stock disponible, posición de inventario, stock de seguridad, punto de reorden, EOQ, costo promedio ponderado, salida por FEFO o PEPS y control de piezas retiradas. También aplica los controles de integración: un dato con un solo dueño, código maestro único, bandeja de salida, idempotencia, reintentos, incidencias y bitácora inalterable. Este documento describe cómo se construyó, qué cubre y la evidencia de que funciona.

1. Objetivo y alcance del desarrollo

El objetivo es demostrar que el sistema de información vincula Mantenimiento, Inventario y Compras sin duplicar datos y sin perder operaciones cuando un sistema externo falla. La tabla 1 separa lo que el prototipo implementa de lo que queda para la puesta en producción.

Tabla 1
Alcance del prototipo

Elemento	En el prototipo	Para producción
Aplicación web por rol	Implementada en React; adaptable a tableta y teléfono	Despliegue con HTTPS en el servidor web de la empresa
API de negocio	Implementada en Node.js 22 y Express 4 con autenticación JWT	Igual; secretos en un gestor de claves
Trabajador de integración	Implementado; transmite la bandeja de salida, reintenta y concilia	Igual
Base de datos	PostgreSQL 16 con 23 tablas, restricciones y disparador de bitácora	Igual, con copia de seguridad diaria
Cola de operaciones	RabbitMQ 3.12: cola principal, reintento con TTL y fallidas	Igual
WMS y ERP	Simuladores con el mismo contrato REST y webhooks	Conexión con los sistemas reales de YPFB Transporte

Elemento	En el prototipo	Para producción
Tablero gerencial	Fuera de alcance (fase 3, Power BI)	Vista de solo lectura sobre PostgreSQL

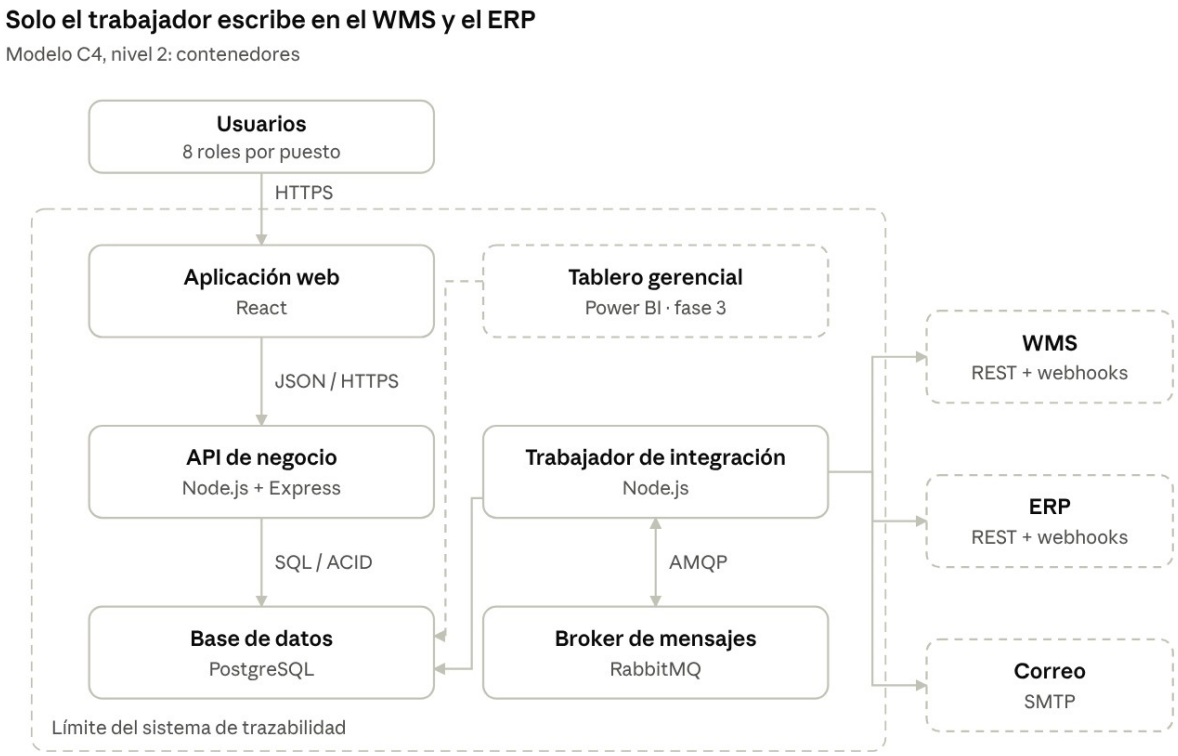
El código fuente, la base de datos, las pruebas y la colección de Postman se entregan en el repositorio del proyecto (carpeta ypfb-mro). La interfaz también se publicó como página de demostración que ejecuta el mismo núcleo de reglas en el navegador, para la presentación ante el docente.

2. Arquitectura implementada

La solución respeta la arquitectura de tres capas definida en la propuesta tecnológica. Cada capa solo se comunica con la contigua: la interfaz no accede a la base de datos y la API no escribe en el WMS ni en el ERP. La figura 1 reproduce el diagrama de contenedores y la tabla 2 indica dónde está cada pieza en el código.

Figura 1

Contenedores implementados (modelo C4, nivel 2)



Nota. Elaboración propia. Corresponde a la figura 3 de la propuesta tecnológica.

Tabla 2

Capas, tecnologías y ubicación en el código

Capa	Pieza	Tecnología	Carpeta
Presentación	Aplicación web por rol	React 18	web/src
Lógica	API de negocio, monolito modular	Node.js 22, Express 4, JWT	api/src
Lógica	Trabajador de integración	Node.js, amqplib	worker
Lógica	Núcleo de reglas: servicios, cálculos, estados, maestros y permisos	JavaScript sin dependencias	core
Datos	Base de datos propia	PostgreSQL 16	db
Datos	Cola de operaciones pendientes	RabbitMQ 3.12	api/src/adaptadores/cola-rabbitmq.js
Sistemas existentes	Simuladores del WMS y del ERP	Express	mocks

2.1 Bajo acoplamiento y alta cohesión

Las reglas del proceso están en un núcleo que no conoce Express, SQL ni HTTP. El núcleo recibe sus dependencias como puertos (interfaces): repositorios de datos, cliente del WMS, cliente del ERP, cola, reloj y generador de identificadores. La raíz de composición (api/src/contenedor.js) es el único lugar donde se eligen las implementaciones concretas. Por eso el mismo núcleo se ejecuta en el servidor con PostgreSQL y en la página de demostración con repositorios en memoria, sin cambiar una línea de las reglas. Esa es la evidencia práctica de bajo acoplamiento.

Tabla 3

Puertos del núcleo y sus adaptadores

Puerto	Adaptador en el servidor	Adaptador en la demostración y en pruebas
Repositorios y transacción	PostgreSQL: db-pg.js (BEGIN, COMMIT, ROLLBACK)	Memoria: repos.js (con reversión ante error)
Cliente del WMS	HTTP: clientes-http.js	Simulador en el mismo proceso
Cliente del ERP	HTTP: clientes-http.js	Simulador en el mismo proceso
Cola de operaciones	RabbitMQ: cola-rabbitmq.js	Sin cola; barrido de la bandeja
Reloj e identificadores	Fecha del sistema y UUID v4	Reloj controlable en las pruebas

La alta cohesión se ve en la organización del núcleo: cada servicio agrupa una sola responsabilidad del proceso (Catálogo, Mantenimiento, Inventario, Compras, Integración, Alertas y Auditoría); todos los cálculos de inventario están juntos en un módulo de funciones puras (core/dominio/calculos.js); todos los estados y transiciones están en un único módulo

(core/dominio/estados.js), y la autorización por rol se aplica dentro de cada servicio, de modo que una llamada directa a la API recibe el mismo rechazo que la pantalla.

2.2 Funcionamiento de Express en la solución

Express recibe la solicitud HTTP, aplica el middleware y la entrega a la ruta del componente correspondiente. La ruta solo traduce HTTP a una llamada al servicio; no contiene reglas. Los errores llegan a un manejador central que responde siempre con el mismo formato. La tabla 4 sigue el recorrido de una reserva.

Tabla 4

Recorrido de una solicitud de reserva en Express

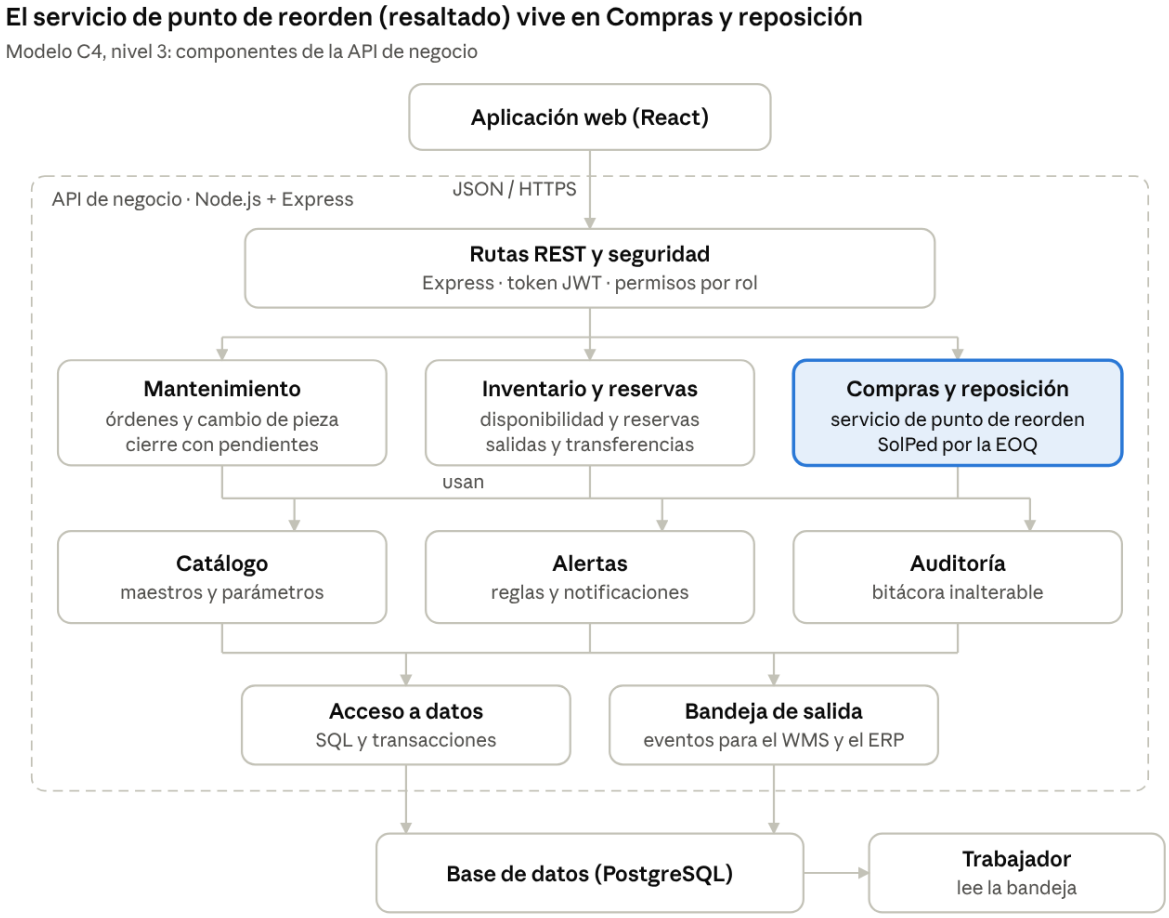
Paso	Qué ocurre	Archivo
1	POST /api/ordenes/OM-2026-0153/reservas llega con el token JWT	api/src/app.js
2	Middleware: identificador de solicitud, lectura de JSON y verificación del token	api/src/middleware/seguridad.js
3	La ruta del componente Mantenimiento llama al servicio	api/src/rutas/index.js
4	El servicio autoriza el rol y valida el estado de la orden	core/servicios/ordenes.js
5	En una transacción se crean la reserva Solicitada y su operación en la bandeja de salida	core/servicios/soporte.js
6	Tras el COMMIT se publica el id de la operación en RabbitMQ	api/src/adaptadores/cola-rabbitmq.js
7	La API responde 202 Accepted: aceptada, todavía no confirmada	api/src/rutas/index.js
8	El trabajador consume el mensaje y envía POST /wms/reservas con Idempotency-Key	worker/index.js
9	Si hay error de negocio, el manejador responde { error: { codigo, mensaje } }	api/src/middleware/errores.js

3. Módulos desarrollados

Los nueve módulos de la propuesta funcional se implementan con los componentes de la API definidos en el diagrama C4 de nivel 3 (figura 2). La tabla 5 relaciona cada módulo con su componente, su código y sus requisitos.

Figura 2

Componentes de la API de negocio (modelo C4, nivel 3)



Nota. Elaboración propia.

Tabla 5

Módulos, componentes y código

Módulo funcional	Componente	Código principal	Requisitos
Datos maestros	Catálogo	core/servicios/maestros.js, core/dominio/maestros.js	RF-01 a RF-03
Usuarios, roles y permisos	Rutas REST y seguridad	core/dominio/permisos.js, api/src/seguridad.js	RF-04
Mantenimiento	Mantenimiento	core/servicios/ordenes.js	RF-05, RF-06, RF-11, RF-12
Inventario y reservas	Inventario y reservas	core/servicios/inventario.js, ordenes.js	RF-07 a RF-10
Compras y reposición	Compras (servicio ROP)	core/servicios/compras.js, webhooks.js	RF-13 a RF-17
Alertas	Alertas	core/servicios/soporte.js, consultas.js	RF-18

Módulo funcional	Componente	Código principal	Requisitos
Trazabilidad y auditoría	Auditoría	core/servicios/consultas.js, tabla bitacora	RF-19, RF-20
Integración de sistemas	Bandeja de salida y trabajador	core/servicios/integracion.js, worker/index.js	RF-21
Sincronización	Trabajador de integración	integracion.js: respaldo y conciliación	RF-22, RF-23

El repositorio suma unas 7 000 líneas de código propio entre núcleo, API, trabajador, simuladores, interfaz, esquema de base de datos y pruebas.

4. Cobertura de requisitos

4.1 Requisitos funcionales

La tabla 6 indica cómo se implementó cada requisito y con qué evidencia se comprueba. Las pruebas se describen en la sección 12; los escenarios de demostración, en la sección 13.

Tabla 6
Requisitos funcionales implementados y su evidencia

Código	Implementación	Evidencia
RF-01	Alta de materiales con código asignado por el maestro, descripción normalizada única y parámetros de reposición	Prueba MDM; escenario de códigos duplicados
RF-02	Equipos con código único y tabla de compatibilidad material–equipo	Prueba de material incompatible
RF-03	Proveedores con NIT único, plazo y porcentaje de cumplimiento	Puntaje de cotizaciones
RF-04	Siete roles y 66 permisos en tablas; JWT; contraseñas con scrypt	Pruebas 401 y 403
RF-05	Orden con equipo, tipo, prioridad, diagnóstico, fecha y materiales compatibles	Flujo principal
RF-06	Validación por el supervisor con fecha y responsable	Flujo principal; prueba de permisos
RF-07	Consulta al WMS de físico, reservado, bloqueado y disponible, más en tránsito del ERP	Pantalla de inventario; prueba HTTP
RF-08	Reserva solo registrada cuando el WMS la confirma	Escenarios de stock insuficiente y última unidad
RF-09	Entrega en ventanilla con receptor; el WMS elige lotes por FEFO o PEPS	Flujo principal; escenario FEFO
RF-10	Liberación de reservas confirmada por el WMS. Devoluciones y transferencias quedan definidas en el contrato para la siguiente iteración	Prueba de liberación (parcial)

Código	Implementación	Evidencia
RF-11	Cambio de pieza exige salida confirmada y registra el destino de la retirada	Flujo principal
RF-12	Cierre bloqueado con la lista de operaciones sin confirmar	Escenario de WMS sin respuesta
RF-13	Evaluación de la posición contra el ROP sin duplicar una compra en curso	Escenario de compra en curso
RF-14	Solicitud por falta de stock ligada a la orden de origen	Escenario de stock insuficiente
RF-15	Cotizaciones comparadas por precio, plazo, garantía y cumplimiento	Prueba HTTP de compra
RF-16	La solicitud aprobada viaja al ERP; la OC se registra solo cuando el ERP la confirma	Prueba HTTP de compra
RF-17	Recepción completa o parcial por webhook; solo lo aceptado incrementa el stock	Escenario de recepción
RF-18	Alertas con tipo, prioridad, destinatario y estado	Pantalla de auditoría
RF-19	Bitácora de solo inserción protegida por disparador en PostgreSQL	Prueba de UPDATE y DELETE rechazados
RF-20	Trazabilidad de la orden: reserva, compra, recepción, salida, instalación y bitácora	GET /api/ordenes/:numero/trazabilidad
RF-21	Bandeja de salida con UUID, Idempotency-Key, consulta por id y reintentos	Escenarios de confirmación perdida e idempotencia
RF-22	Incidencias con código, causa, responsable y estado; filtro por código y estado	Escenario de WMS sin respuesta
RF-23	Conciliación de existencias WMS contra ERP con registro de diferencias. La sincronización automática del catálogo queda para producción	Escenario de conciliación (parcial)

4.2 Requisitos no funcionales

Tabla 7

Requisitos no funcionales y su comprobación en el prototipo

Código	Atributo	Comprobación
RNF-01	Rendimiento	Tiempo medio de respuesta de la API en Postman: 20 ms (máximo 98 ms). Una operación normal se confirma en el siguiente mensaje de la cola, en menos de 1 s
RNF-02	Disponibilidad	Ruta /salud y reinicio de contenedores con Docker Compose; la medición mensual corresponde a producción
RNF-03	Recuperación	Cola en RabbitMQ, reintentos 2^n, barrido de la bandeja cada 5 s y sincronización de respaldo cada 15 min
RNF-04	Integridad	El mismo id enviado tres veces produce una sola reserva en el WMS

Código	Atributo	Comprobación
RNF-05	Consistencia	Transacciones ACID; FOR UPDATE SKIP LOCKED; bloqueo por material; prueba de dos reservas simultáneas
RNF-06	Seguridad	JWT con expiración, contraseñas con scrypt, autorización por rol en el servicio, webhooks firmados con HMAC-SHA256; HTTPS en el despliegue
RNF-07	Auditabilidad	Bitácora con usuario, rol, fecha, acción, valor anterior, valor nuevo y motivo
RNF-08	Usabilidad	Interfaz adaptable comprobada a 400 px de ancho
RNF-09	Respaldo	Procedimiento pg_dump diario en producción; fuera del prototipo
RNF-10	Exactitud	Conciliación WMS contra ERP; el conteo semanal es un procedimiento operativo

5. Interfaces y pantallas

La aplicación presenta las funciones por rol. Cada botón indica el rol que puede ejecutar la acción y el servicio vuelve a verificarlo. Para la demostración, la opción "Usar el rol de cada acción" ejecuta cada paso con el usuario que corresponde; al desactivarla, el usuario elegido en "Actuar como" recibe el rechazo SIN_PERMISO si no tiene el permiso.

Tabla 8
Interfaces por rol

Rol	Funciones disponibles
Técnico de mantenimiento	Crear órdenes con materiales compatibles, registrar el cambio de pieza y completar la intervención
Supervisor de mantenimiento	Validar y priorizar órdenes, revisar operaciones pendientes y cerrar la orden
Personal de almacén	Consultar disponibilidad, solicitar reservas, registrar entregas y liberar reservas
Comprador	Evaluar reposición, registrar y comparar cotizaciones, aprobar el envío al ERP
Control interno	Consultar bitácora, trazabilidad, incidencias y conciliaciones sin modificar registros
Administrador	Administrar el catálogo, reprocesar operaciones y ejecutar la conciliación
Gerencia	Consultar órdenes, compras e incidencias sin modificar información operativa

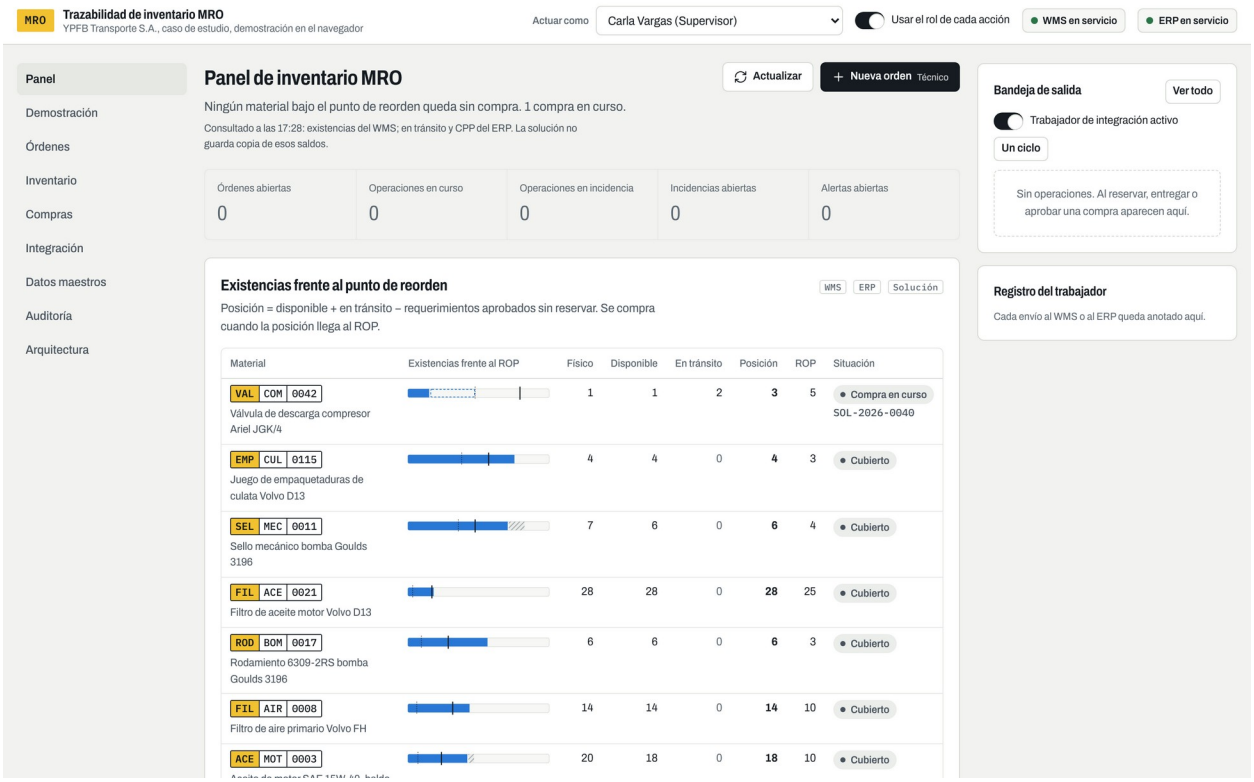
Tabla 9
Pantallas desarrolladas

Pantalla	Información y controles
Panel	Estado del proceso y existencias de cada material frente a su ROP, con la fuente de cada dato
Órdenes	Lista, nueva orden con materiales compatibles, recorrido de estados, acciones por rol, reservas, salidas, cambio de pieza y operaciones de integración
Inventario	Disponibilidad del WMS, posición, fórmulas con los valores del material, lotes en orden de salida y kardex valorizado del ERP

Pantalla	Información y controles
Compras	Evaluación contra el ROP, solicitudes, cotizaciones con puntaje, aprobación, OC del ERP y recepción
Integración	Simulación de fallas, bandeja de salida con intentos y causa, contadores del WMS, incidencias y conciliación
Datos maestros	Alta de material con código propuesto y detección de duplicados, catálogo, compatibilidad y proveedores
Auditoría	Alertas con destinatario y bitácora de solo lectura
Demostración	Escenarios que responden las preguntas de la defensa ejecutando el sistema
Arquitectura	Diagrama de la solución, dueño de cada dato, recorrido en Express y rutas de la API

Figura 3

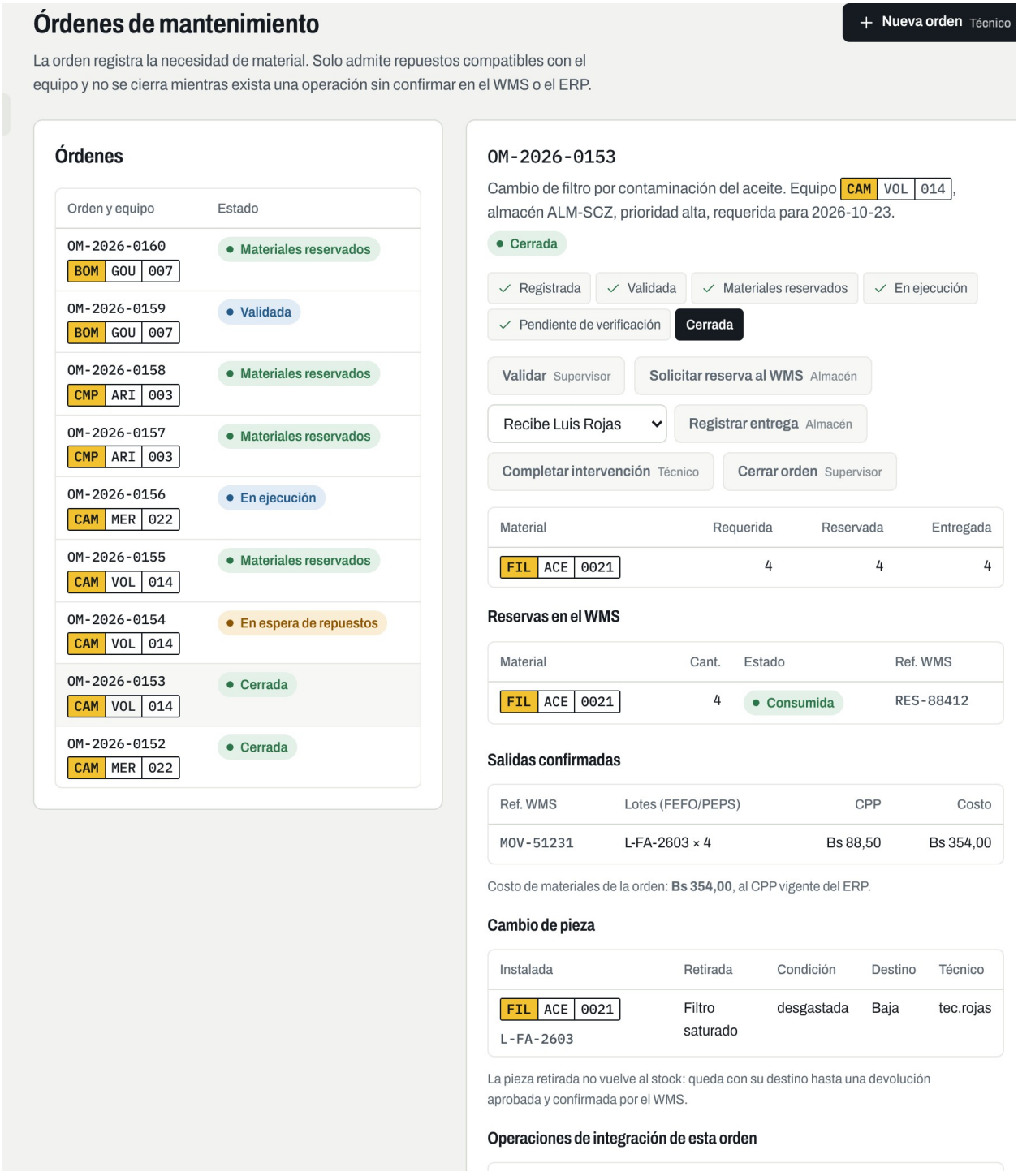
Pantalla Panel: existencias de cada material frente a su punto de reorden



Nota. Elaboración propia. Captura del sistema en funcionamiento. Las etiquetas WMS, ERP y Solución indican el dueño de cada dato.

Figura 4

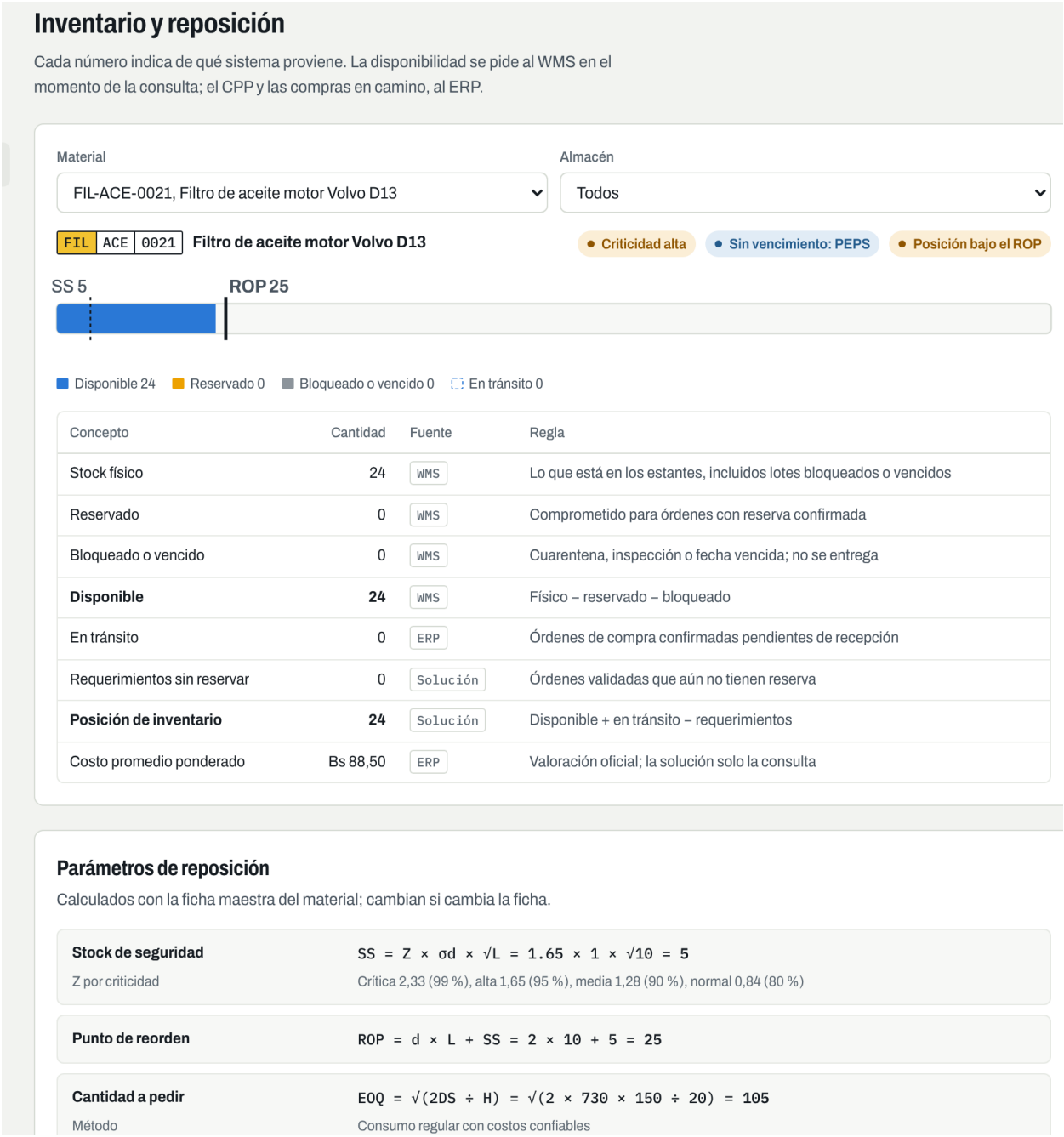
Orden OM-2026-0153 cerrada: reserva, salida, costo al CPP y cambio de pieza



Nota. Elaboración propia. Captura del sistema en funcionamiento.

Figura 5

Pantalla Inventario: disponibilidad, fuentes y fórmulas de reposición de FIL-ACE-0021



Nota. Elaboración propia. Captura del sistema en funcionamiento.

6. Flujo funcional desarrollado

Cada transición depende de un evento confirmado. La tabla 10 muestra la secuencia principal con la ruta de la API que la ejecuta.

Tabla 10
Secuencia principal y rutas de la API

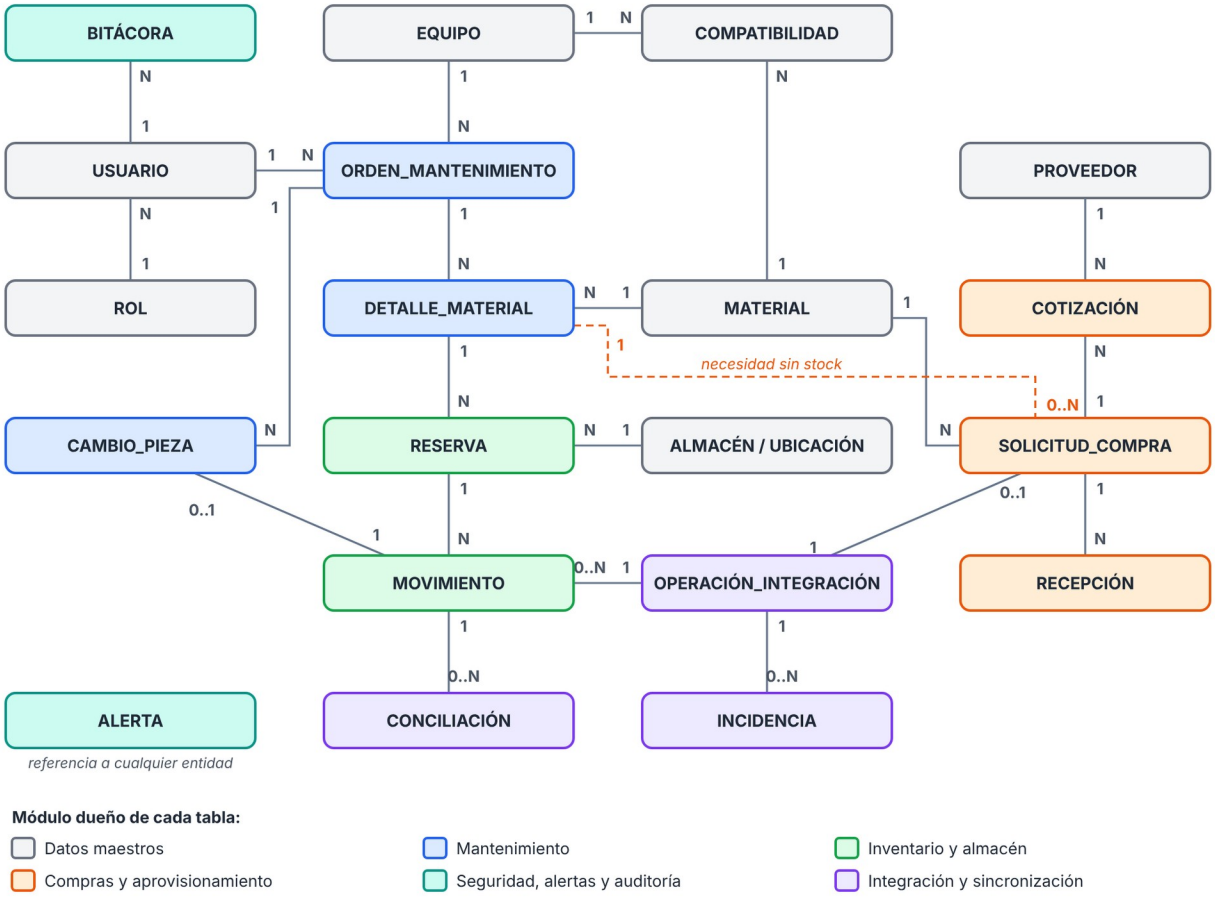
Etapa	Ejecución y resultado	Ruta
1 Necesidad	El técnico crea la orden con materiales compatibles; estado Registrada	POST /api/ordenes
2 Validación	El supervisor confirma prioridad y responsable; estado Validada	POST /api/ordenes/:numero/validacion
3 Consulta	El conector obtiene del WMS físico, reservado, bloqueado y disponible	GET /api/inventario/disponibilidad/:material
4 Reserva	La reserva queda en la bandeja; el WMS la confirma; estado Materiales reservados	POST /api/ordenes/:numero/reservas
5 Compra	Sin stock, la orden pasa a En espera de repuestos y se crea la solicitud ligada	Automático; POST /api/compras/solicitudes/:numero/aprobacion
6 Recepción	El WMS acepta el material y avisa por webhook; se reintenta la reserva	POST /api/integracion/webhooks/wms
7 Entrega	La salida se confirma en el WMS; se valoriza al CPP y se evalúa el ROP	POST /api/ordenes/:numero/entregas
8 Intervención	El técnico registra la pieza instalada y la retirada con su destino	POST /api/ordenes/:numero/cambios-pieza
9 Cierre	El supervisor cierra si no hay operaciones pendientes	POST /api/ordenes/:numero/cierre
10 Control	Bitácora, alertas e incidencias conservan la evidencia	GET /api/ordenes/:numero/trazabilidad

Las transiciones se aplican con las máquinas de estado de la fase de modelado (core/dominio/estados.js). Una transición no prevista, por ejemplo cerrar una orden validada, se rechaza con el código TRANSICION_INVALIDA.

7. Modelo de datos aplicado al desarrollo

El esquema db/schema.sql implementa las 23 tablas del modelo entidad-relación de la fase de modelado (figura 6). La solución guarda sus propios objetos y referencias a los objetos oficiales: no guarda saldos de stock ni órdenes de compra formales. Una operación de integración puede no tener movimiento ni solicitud asociados, o tener varios; por eso la relación se mantiene 1:0..N, como en el modelado.

Figura 6
Modelo entidad-relación implementado en PostgreSQL



Nota. Elaboración propia. Corresponde a la figura 6 de la fase de modelado; el esquema está en db/schema.sql.

Las reglas de integridad del diseño no dependen solo de la aplicación: la base de datos también las impone, como muestra la tabla 11.

Tabla 11
Reglas de integridad y su control en PostgreSQL

Regla	**Control en la base de datos**
Un material, un código	Clave primaria con formato FAM-SUB-NNNN y descripción normalizada UNIQUE
Una sola compra en curso por material	Índice único parcial ux_solicitud_abierta
Una reserva activa por detalle de orden	Índice único parcial ux_reserva_activa
Lo reservado y entregado no supera lo requerido	CHECK sobre detalle_material
Lo aceptado no supera lo recibido	CHECK sobre recepcion
Todo cambio de pieza referencia una salida confirmada	movimiento_id NOT NULL con clave foránea

Regla	Control en la base de datos
Cada operación tiene un identificador único	Clave primaria UUID en operacion_integracion
La bitácora no se modifica	Disparador que rechaza UPDATE, DELETE y TRUNCATE
Estados válidos	CHECK con la lista de estados de cada máquina

8. Lógica del inventario y reposición

El inventario es MRO, perpetuo, por almacén y por criticidad. La tabla 12 muestra los valores que el sistema calcula para el filtro de aceite FIL-ACE-0021, con los datos de la ficha maestra (d = 2 por día, σd = 1, L = 10 días, criticidad alta, D = 730, S = Bs 150, H = Bs 20). Son los mismos valores del ejemplo de la fase conceptual y se verifican en una prueba automatizada.

Tabla 12
Valores calculados por el sistema para FIL-ACE-0021

Valor	Cálculo	Resultado	Fuente
Stock disponible	Físico − reservado − bloqueado = 28 − 4 − 0	24	WMS
Posición de inventario	Disponible + en tránsito − requerimientos sin reservar = 24 + 0 − 0	24	Solución
Stock de seguridad	SS = Z × σd × √L = 1,65 × 1 × √10	5	Solución
Punto de reorden	ROP = d × L + SS = 2 × 10 + 5	25	Solución
Cantidad a comprar	EOQ = √(2 × 730 × 150 ÷ 20)	105	Solución
Costo de la salida	4 unidades × CPP Bs 88,50	Bs 354,00	ERP

Después de cada salida confirmada el sistema compara la posición con el ROP. En la demostración, la orden OM-2026-0153 retiró 4 filtros, la posición quedó en 24, igual o menor al ROP de 25, y el sistema creó la solicitud SOL-2026-0041 por 105 unidades. Para materiales de consumo irregular se compra hasta el máximo: la empaquetadura EMP-CUL-0115 tenía posición −2 (4 disponibles y una orden de 6) y la solicitud fue de 5 − (−2) = 7 juegos.

8.1 Valoración por costo promedio ponderado

El ERP recalcula el CPP en cada ingreso aceptado; la solución lo consulta para valorizar el consumo. En la demostración, la válvula VAL-COM-0042 tenía 1 unidad a Bs 6 850 e ingresó 1 unidad de la OC 4500012871 a Bs 6 900:

$$\mathbf{CPP = (1 \times 6\,850 + 1 \times 6\,900) \div 2 = Bs\ 6\,875}$$

8.2 Selección de unidades

Primero se valida la compatibilidad con el equipo, en la orden. Después el WMS aplica FEFO a los materiales con vencimiento y PEPS a los demás. En la demostración, una orden de 3 baldes de aceite ACE-MOT-0003 recibió el lote L-2401 (vence el 15/12/2026) antes que el L-2405 (vence el 30/06/2027); el lote L-2312, vencido el 01/10/2026, figura como bloqueado y no se entrega. El filtro FIL-ACE-0021, sin vencimiento, salió del lote más antiguo, L-FA-2603.

8.3 Piezas retiradas

La pieza retirada no vuelve al stock utilizable. El cambio de pieza exige su condición (desgastada, dañada, en garantía o reutilizable) y su destino (evaluación, reparación, garantía, resguardo o baja). Solo una devolución aprobada y confirmada por el WMS puede reincorporar material al inventario.

9. Datos maestros y control de duplicidad

Un mismo repuesto no puede tener dos códigos. El código lo asigna el maestro con la estructura familia-subfamilia-correlativo (FIL-ACE-0021: filtros, aceite, número 21) y lo usan Mantenimiento, el WMS y el ERP. Antes de registrar un material, la descripción se normaliza: minúsculas, sin tildes ni signos, sin palabras vacías, con sinónimos unificados y palabras ordenadas. Si la similitud con un material existente es de 75 % o más, el alta se rechaza con MATERIAL_DUPLICADO.

Tabla 13

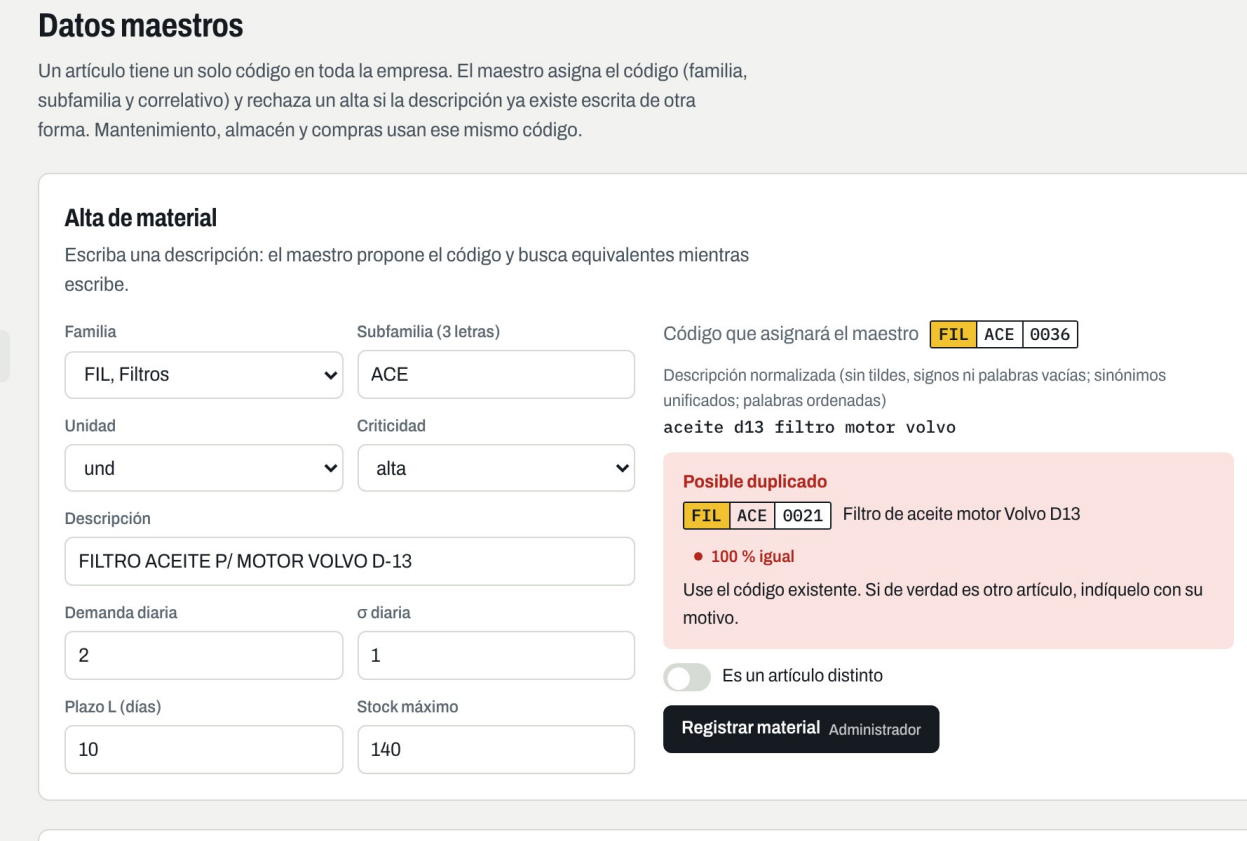
Ejemplo de detección de duplicados

Descripción ingresada	Descripción normalizada	Resultado
Filtro de aceite motor Volvo D13 (existente)	aceite d13 filtro motor volvo	Código FIL-ACE-0021
FILTRO ACEITE P/ MOTOR VOLVO D-13	aceite d13 filtro motor volvo	Rechazado: 100 % igual a FIL-ACE-0021
Filtro de aceite motor Scania DC13	aceite dc13 filtro motor scania	Aceptado: código FIL-ACE-0035

La base de datos es la segunda barrera: la descripción normalizada tiene restricción UNIQUE, de modo que ni una llamada directa ni un error del programa pueden crear el duplicado. La figura 7 muestra la pantalla de alta con la detección mientras se escribe.

Figura 7

Alta de material: código propuesto y rechazo del duplicado



Nota. Elaboración propia. Captura del sistema en funcionamiento.

Fuera del catálogo, la duplicidad se evita asignando cada dato a un único dueño: el stock se consulta al WMS en cada operación, el CPP y la OC al ERP, y la solución guarda solo referencias (número de reserva, de movimiento y de OC). La pantalla de inventario identifica la fuente de cada número.

10. Integración con WMS y ERP

Tabla 14

Mecanismos de integración implementados

Mecanismo	Uso	Implementación
API REST síncrona	Consulta de disponibilidad, en tránsito, valoración y kardex	GET /wms/disponibilidad, /erp/en-transito, /erp/valoracion, /erp/kardex
Cola asíncrona	Reservas, salidas, liberaciones y solicitudes al ERP	Bandeja de salida en PostgreSQL y cola mro.operaciones en RabbitMQ
Webhooks	Recepción aceptada, transferencia vencida y OC reprogramada	POST /api/integracion/webhooks/wms y /erp con firma HMAC
Proceso programado	Respaldo y conciliación	Sincronización cada 15 min; POST /api/integracion/conciliaciones

10.1 Contrato de reserva

El trabajador envía la reserva con el id de la operación como clave de idempotencia. Si el mensaje se reenvía, viaja con el mismo id y el WMS devuelve la misma reserva sin crear otra.

```
POST /wms/reservas
Idempotency-Key: 7f3c9a1e-2b4d-4e8f-9a61-1c2d3e4f5a6b

{ "id_operacion": "7f3c9a1e-2b4d-4e8f-9a61-1c2d3e4f5a6b",
  "orden": "OM-2026-0153", "material": "FIL-ACE-0021",
  "almacen": "ALM-SCZ", "cantidad": 4 }

201 { "resultado": "confirmada", "ref_reserva": "RES-88412" }
409 { "resultado": "rechazada", "causa": "STOCK_INSUFICIENTE", "disponible": 2 }
```

Antes de reenviar una operación que no tuvo respuesta, el trabajador consulta GET /wms/operaciones/{id}. Si el WMS ya la aplicó, se usa esa respuesta.

10.2 Contrato del webhook de recepción

```
POST /api/integracion/webhooks/wms
X-Firma: HMAC-SHA256 del cuerpo

{ "id_evento": "uuid", "evento": "recepcion.aceptada",
  "orden_compra": "4500012871", "material": "VAL-COM-0042",
  "cantidad_recibida": 1, "cantidad_aceptada": 1, "lote": "L-VD-2610" }
```

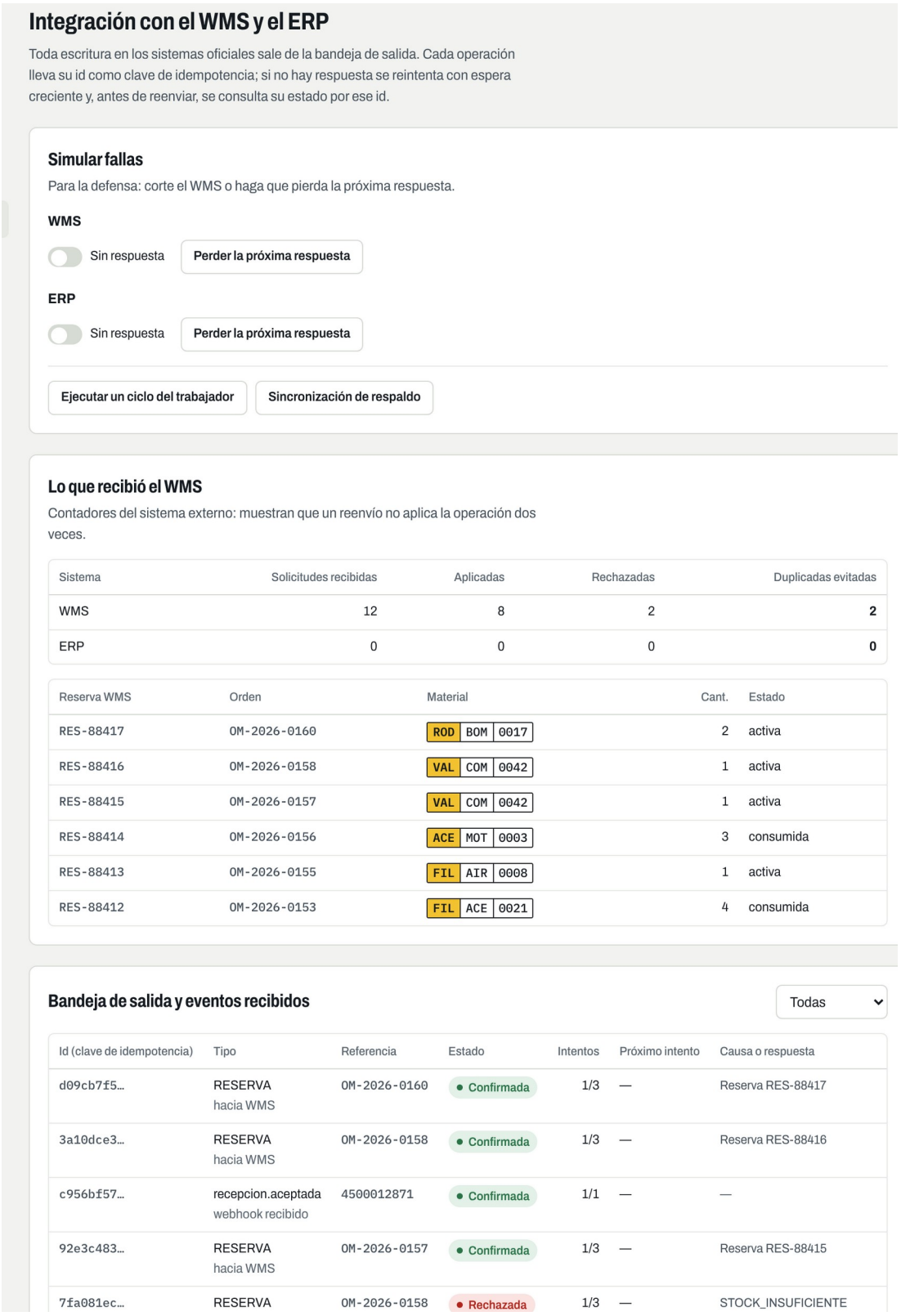
El id_evento se registra en operacion_integracion como evento entrante: si el mismo evento llega dos veces, se aplica una sola vez.

10.3 Cola en RabbitMQ

La bandeja de salida en PostgreSQL es la fuente de verdad y RabbitMQ transporta el aviso. La operación se guarda en la misma transacción que el cambio de negocio; después del COMMIT se publica su id en mro.operaciones. Si no hay respuesta, el trabajador publica el mensaje en mro.operaciones.reintento con un tiempo de vida igual a la espera (2 s y luego 4 s); al vencer, RabbitMQ lo devuelve a la cola principal. Al agotar los intentos, el mensaje pasa a mro.operaciones.fallidas y se registra la incidencia. Si un aviso no llega al broker, el barrido de la bandeja cada 5 s recupera la operación.

Figura 8

Pantalla Integración: simulación de fallas, contadores del WMS y bandeja de salida



Nota. Elaboración propia. Captura del sistema en funcionamiento.

11. Sincronización y recuperación ante fallas

El envío de una operación no equivale a una operación completada. La tabla 15 resume la respuesta implementada para cada falla y su evidencia.

Tabla 15

Tratamiento de fallas implementado

Situación	Respuesta del sistema	Evidencia
WMS sin respuesta	Reintentos a 1,5 s y 3 s (demostración) o a 2 s y 4 s (servidor); al tercer intento fallido; incidencia WMS_SIN_RESPUESTA y alerta a Almacén	Escenario y prueba de caída
ERP sin respuesta	Igual, con ERP_SIN_RESPUESTA y alerta a Compras	Misma lógica por destino
Stock insuficiente	Rechazo sin tocar el stock, orden en espera y solicitud de compra	Escenario de stock insuficiente
Datos inválidos	Rechazo con DATOS_INVALIDOS sin modificar información	Prueba de material inexistente
Confirmación perdida	Consulta por id antes de reenviar; incidencia CONFIRMACION_PERDIDA ya resuelta	El WMS recibió un solo envío
Recepción parcial	Ingresa lo aceptado; la solicitud queda Recibida parcial con saldo y alerta	Escenario de recepción
Proveedor retrasado	Webhook oc.reprogramada: nueva fecha, incidencia y alerta a Compras	POST /simulador/erp/reprogramaciones
Diferencia de conciliación	Registro de la diferencia e incidencia para Almacén	Escenario de conciliación

Figura 9

Escenario de demostración: WMS sin respuesta y recuperación sin duplicar



Nota. Elaboración propia. Captura del sistema en funcionamiento.

Los controles técnicos recuperan operaciones que sí fueron registradas; no pueden crear un movimiento físico que nunca se registró. Por eso todo movimiento se registra al momento de realizarse y la conciliación detecta los ajustes hechos fuera del proceso.

12. Validación funcional y técnica

La validación se hizo en tres niveles. Las pruebas unitarias ejecutan el núcleo en memoria con un reloj controlado. Las pruebas de integración levantan la API Express, PostgreSQL 16, RabbitMQ y

los simuladores, y recorren el proceso por HTTP. La colección de Postman se ejecutó con Newman contra el sistema completo en marcha.

Tabla 16

Resultados de las pruebas automatizadas

Conjunto	**Qué ejecuta**	**Resultado**
Unitarias (node:test)	Cálculos, estados, MDM y flujos del núcleo en memoria	23 de 23 aprobadas
Integración (node:test)	API Express, PostgreSQL, RabbitMQ y simuladores por HTTP	12 de 12 aprobadas
Postman (Newman)	33 solicitudes en orden sobre el sistema en marcha	42 de 42 aserciones; 20 ms de respuesta media

Tabla 17

Escenarios de validación y resultado obtenido

Escenario	**Resultado obtenido**
Reserva y salida con stock suficiente	OM-2026-0153: RES-88412 confirmada, salida MOV-51231 del lote L-FA-2603, Bs 354,00
Cantidad superior al disponible	STOCK_INSUFICIENTE; stock sin cambios; SOL-2026-0042 ligada a OM-2026-0154
Envío repetido de la misma solicitud	Tres envíos del mismo id; una sola reserva RES-88417
Interrupción antes del procesamiento	Dos reintentos, incidencia, cierre bloqueado y reproceso confirmado
Pérdida de confirmación	Segundo intento recuperado por id; el WMS recibió un solo envío
Dos solicitudes simultáneas	Una orden en Materiales reservados y la otra En espera de repuestos
Punto de reorden alcanzado	SOL-2026-0041 por 105 unidades (EOQ)
Punto de reorden con compra en curso	No se crea otra solicitud: SOL-2026-0040 con OC 4500012871
Recepción parcial	1 de 2 unidades; CPP de Bs 6 850 a Bs 6 875; reserva reintentada
Rol sin permiso	SIN_PERMISO; la bitácora no registra cambios
Material duplicado	MATERIAL_DUPLICADO por la API y DUPLICADO por la base de datos
Bitácora	UPDATE y DELETE rechazados por el disparador
Conciliación	ROD-BOM-0017: WMS 5 y ERP 6; incidencia DIFERENCIA_CONCILIACION

Figura 10

Escenario de demostración: confirmación perdida y salida por FEFO



Nota. Elaboración propia. Captura del sistema en funcionamiento.

12.1 Calidad de la interfaz

La interfaz se construyó con cuatro skills de diseño (frontend-design, artifact-design, artifact-diagramming y dataviz) y se revisó con tres más, descargadas de sus repositorios públicos: taste-skill, impeccable y ui-ux-pro-max. Cada una aporta un medidor que se ejecutó sobre la aplicación en un navegador real a 1 440, 768 y 390 píxeles de ancho.

Tabla 18

Revisión de la interfaz con las skills de diseño

Herramienta	Antes	Después
taste-skill: lint de pantallas de aplicación	2 fallas y 7 avisos	0 fallas; 2 avisos justificados
taste-skill: inspector en el navegador, 9 vistas	1 falla y 1 aviso en el panel	0 fallas en las 9 vistas
impeccable: detector de patrones genéricos	3 hallazgos	0 hallazgos

Herramienta	Antes	Después
impeccable: auditoría técnica	sin medir	19 de 20 (excelente)
ui-ux-pro-max: lista de verificación de entrega	sin medir	7 de 7 puntos

Los cambios principales fueron: la interfaz en tinta con color solo en los datos y los estados; un juego de iconos de trazo en lugar de símbolos de texto; indicadores del proceso en un contenedor dividido; el hallazgo del día calculado de los datos bajo el título del panel; filas seleccionables operables con teclado; botones deshabilitados legibles y con su motivo, y objetivos táctiles de 44 píxeles en teléfono. El detalle, con capturas antes y después, está en docs/calidad-interfaz del repositorio.

13. Guía de la demostración

La sección Demostración de la aplicación contiene un escenario por cada pregunta prevista en la defensa. Cada escenario ejecuta el sistema y muestra los datos que produce. La tabla 19 sirve como guion.

Tabla 19

Preguntas de la defensa, escenario y concepto

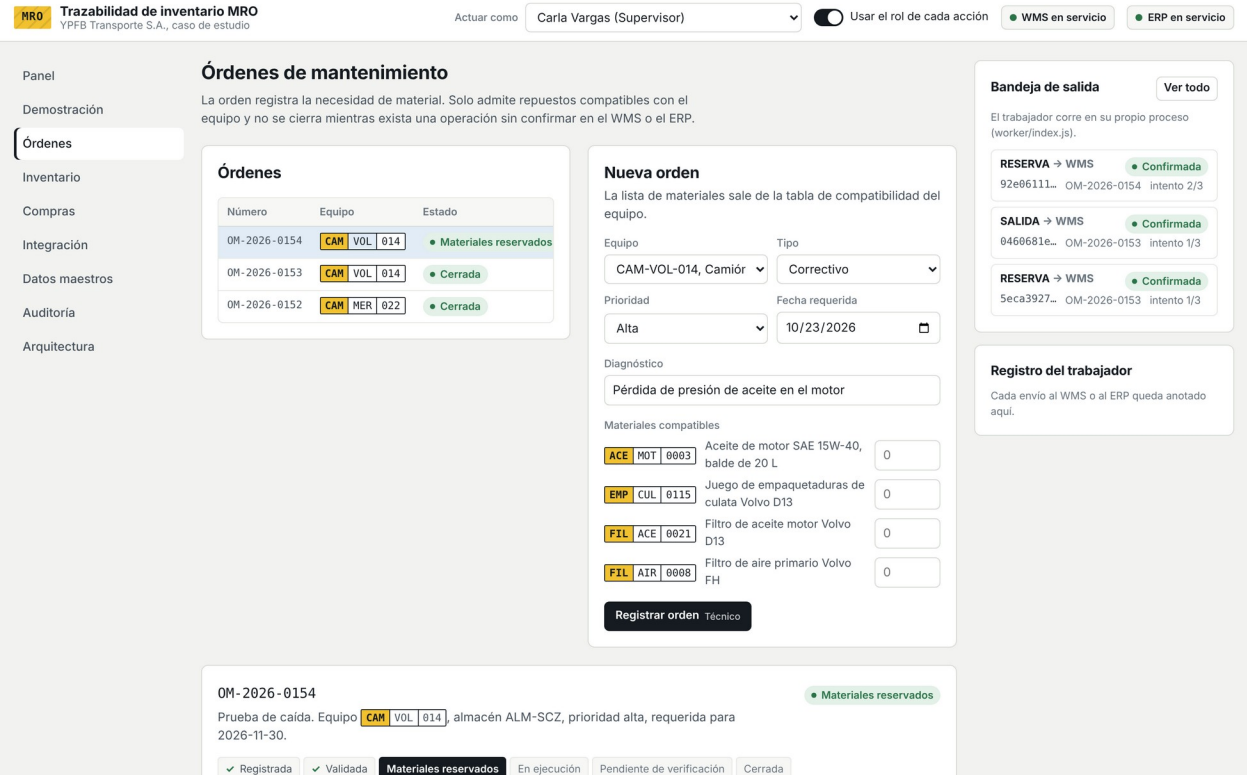
Pregunta	Escenario o pantalla	Concepto que se demuestra
¿Cómo es su inventario y qué algoritmo usan?	Flujo de punta a punta; Inventario	Inventario MRO perpetuo, disponible, posición, SS, ROP, EOQ, CPP, FEFO y PEPS
¿Cómo se integran los módulos?	Arquitectura; Integración	Tres capas, bandeja de salida, RabbitMQ, REST y webhooks
¿Cómo evitan la duplicidad?	Códigos duplicados; compra en curso; idempotencia	Código maestro único, un dato un dueño, mismo id sin doble efecto
¿Cómo manejan los precios?	Inventario, kardex; Compras	CPP calculado por el ERP; precio de la OC; puntaje de cotizaciones
¿Cómo se actualizan los datos?	Llegada de la compra; bandeja de salida	Webhook de recepción, CPP recalculado, reserva reintentada
¿Qué pasa si falla un sistema?	WMS sin respuesta; respuesta perdida	Reintentos, incidencia con causa, consulta por id, cierre bloqueado
¿Hay bajo acoplamiento y alta cohesión?	Arquitectura	Núcleo con puertos; la misma lógica en navegador y servidor
¿Cómo trabaja Express?	Arquitectura: recorrido de una solicitud	Middleware, rutas por componente, servicio y manejador de errores
¿Cómo funcionan los maestros?	Datos maestros	Código asignado, descripción normalizada, compatibilidad y proveedores
¿Cada usuario hace solo lo suyo?	Permisos; Auditoría	Autorización por rol y bitácora inalterable

14. Ejecución del sistema

Con Docker, la orden docker compose up --build levanta PostgreSQL, RabbitMQ, los simuladores (puerto 4000), la API con la interfaz (puerto 3000) y el trabajador. Sin Docker, se crea la base con npm run db:init y se inician npm run mocks, npm run api y npm run worker. Los usuarios de demostración usan la contraseña Demo2026!: tec.rojas, sup.vargas, alm.mamani, com.flores, ctl.quiroga, adm.sistema y ger.salazar. Las pruebas se ejecutan con npm test y npm run test:integracion; la colección está en postman/YPFB-MRO.postman_collection.json.

Figura 11

Interfaz React servida por la API Express con PostgreSQL y RabbitMQ



Nota. Elaboración propia. Captura de la aplicación en http://localhost:3000.

15. Resultado de la fase de desarrollo

El sistema desarrollado registra la necesidad de material, obtiene la disponibilidad oficial del WMS, controla la reserva y la entrega, valoriza el consumo al CPP del ERP, genera solicitudes de compra cuando la posición alcanza el punto de reorden o falta stock, recibe del ERP la referencia de la orden formal y documenta la pieza retirada e instalada.

La consistencia se mantiene porque cada dato tiene un único dueño y cada operación un único identificador. PostgreSQL conserva los objetos propios del proceso y su trazabilidad, RabbitMQ transporta las operaciones pendientes con sus reintentos, el trabajador de integración centraliza la comunicación con los sistemas externos y la API aplica las reglas de compatibilidad, reposición, autorización, cierre e integridad.

El prototipo cubre 21 de los 23 requisitos funcionales de forma completa y 2 de forma parcial (RF-10 y RF-23), y comprueba 8 de los 10 requisitos no funcionales; RNF-02 y RNF-09 corresponden a

la operación en producción. Las 35 pruebas automatizadas y las 42 aserciones de Postman se aprobaron sin fallas. La siguiente fase, de integración y orquestación, parte de estos contratos y de esta evidencia.

Referencias

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Express. (2024). *Express 4.x API reference*. https://expressjs.com/en/4x/api.html

Hohpe, G., y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

International Accounting Standards Board. (2003). *Norma Internacional de Contabilidad 2: Inventarios*. IFRS Foundation.

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

RabbitMQ. (2024). *Time-to-live and expiration; Dead letter exchanges*. https://www.rabbitmq.com/docs/ttl

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Silver, E. A., Pyke, D. F., y Thomas, D. J. (2017). *Inventory and production management in supply chains* (4.ª ed.). CRC Press.